Agent-HAnS: Maintaining Feature and Code Traceability in Agentic Development

Johan Martinson✉
Ruhr University Bochum
Bochum, Germany
Centiro Solutions AB
Borås, Sweden

Jacob Bengtsson
Chalmers University of Technology
Gothenburg, Sweden

Moltas Hultin
Chalmers University of Technology
Gothenburg, Sweden

Kevin Hermann
Ruhr University Bochum
Bochum, Germany

Thorsten Berger
Ruhr University Bochum
Bochum, Germany
Chalmers | University of Gothenburg
Gothenburg, Sweden

Abstract

Agentic coding tools using LLMs enable developers to quickly generate software. However, developers quickly lose the overview over their increasingly complex codebases. During code generation, no knowledge about what features exist or where they are implemented is documented. We present Agent-HAnS, an agentic workflow which instructs the LLM to update a feature model and generate embedded feature annotations after each code-generation step. It visualizes the evolving structure on a live dashboard served via a Model Context Protocol server. We demonstrate Agent-HAnS on a C#/.NET 9 web API evolved through 27 prompts across 10 evolution steps, with most annotations maintained automatically and no active annotation effort from the developer. In a manual audit, 92% of fragment annotations and 70% of file mappings remain correct, with errors concentrating at feature merges and splits. In a user study, Agent-HAnS scored 73.9 on the System Usability Scale.

Tool video: https://youtu.be/Nb-eDrGokI8

CCS Concepts

• **Software and its engineering** → **Software product lines**; • **Computing methodologies** → *Natural language generation.*

Keywords

feature models, feature traceability, agentic coding

ACM Reference Format:
Johan Martinson, Jacob Bengtsson, Moltas Hultin, Kevin Hermann, and Thorsten Berger. 2026. Agent-HAnS: Maintaining Feature and Code Traceability in Agentic Development. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3832783.3834652

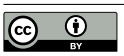
1 Introduction

Features form the focal point of feature-driven software development processes, such as Scrum or XP [5, 8, 16, 17]. They enable developers to organize and reason about the functionalities of a software system that stakeholders seek. Developers have to frequently add, maintain, or evolve features to meet user needs.

Nowadays, developers use agentic coding [12, 23, 30] with tools such as Claude Code, OpenCode, or GitHub Copilot to quickly add or make changes to features by allowing them to prompt Large Language Models (LLM) in natural language. Developers stay in the prompt window, reviewing each generated changeset rather than authoring code line by line [7]. However, *developers quickly lose overview over agentic coding dialogues and what features are added, modified or deleted, as these tools do not maintain an overview of the features in the system across coding cycles* [11, 18, 28].

We advocate making features and their traces to code explicit [2, 6, 15] to enhance the comprehension of agentic coding dialogs and code, as well as to support their maintenance and evolution. Features can be organized in a feature model, which is a hierarchical structure that captures features and their relationships [16, 34]. Maintaining traceability links between feature model and implementation assets assists developers in code comprehension [6, 7, 15, 22, 35], change-impact reasoning [6], and feature location [10, 29]. Feature location—finding where a specific feature is implemented—is already among the most time-consuming activities in software maintenance [10, 26, 29]. Recovering traceability links after features have been implemented is costly and error-prone [27, 33], and automatic techniques produce too many false positives and do not scale to large codebases [29]. Maintaining traceability links while implementing features is more effective and efficient in the long run [6, 20]. Although manual techniques such as embedded feature annotations [22, 31] support maintaining traceability links, *they focus on manual coding workflows and are neither designed for agentic coding nor do they maintain a feature model along traceability links in agentic coding cycles.*

We present Agent-HAnS, an agentic coding workflow comprising a workflow specification, two agent skills, an MCP server, and a dashboard, designed to maintain a feature model and embedded feature annotations across agentic coding cycles. After each code generation step, Agent-HAnS updates the feature model and maintains annotations as part of the coding workflow, with no additional action required from the developer between cycles. It allows developers to maintain an up-to-date feature model and traceability links between features and code. A web dashboard, refreshed after each prompt, visualizes the evolving feature model, maps features to source files, and shows a change summary similar to a git diff at the feature level.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834652

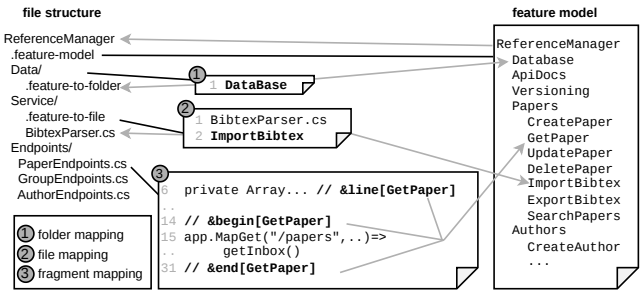


Figure 1: Embedded feature annotation system [31]

Our contributions are threefold: *(i)* **Agent-HAnS**, which maintains a feature model and embedded feature annotations throughout agentic coding, *(ii)* a demonstration on **ReferenceManager**, a C# /.NET 9 web API evolved through 27 prompts across 10 steps (Sect. 4), and *(iii)* a **public dataset** accompanying the demonstration, comprising the annotated ReferenceManager source code at each of the 10 evolution steps, the 27 prompts that drove the agent, and its feature model. The dataset is version controlled and can be used to reproduce the evolution steps of our demonstration.

2 Challenges & Background

Challenge 1: Maintaining Feature Models across Coding Cycles. Agentic coding tools hold no persistent model of the project's feature structure across sessions [11]. A *feature model* organizes a system's features into a tree that captures existing features and their relationships. The feature model evolves as features are added, modified, or removed. Each coding cycle may introduce new features, reorganize existing ones, or remove them without updating the feature model, which accumulates costs over time [6]. The challenge is to automatically maintain a feature model across coding cycles, with minimal developer action required.

So, Agent-HAnS must instruct the agent to maintain the feature model after each coding cycle. This can be achieved by instructing the agent with agent skills to update the model based on the changes to the codebase. Since agents cannot guarantee the correctness of the updated model, for example by hallucinating features or merging existing ones incorrectly, Agent-HAnS must also provide a dashboard for developers to review the feature model after each cycle, such as a change summary or a visual diff of the model. Keeping the human-in-the-loop allows developers to maintain accurate feature models across coding cycles while minimizing maintenance effort.

Challenge 2: Maintaining Traceability Links at Generation Time. Maintaining traceability links while features are fresh in the developer's mind saves feature-location effort compared to retroactive recovery [6, 15]. However, agentic coding shifts this responsibility from developers to agents. The agent, not the developer, holds the model of what each implementation asset contains, but discards it at the end of coding cycles.

So, traceability links must be maintained by Agent-HAnS as code is generated. Since agents are unable to give guarantees on the correctness of these links, Agent-HAnS must show a change summary, so developers can review them before the next coding cycle. It uses embedded feature annotations to maintain traceability links [31], however, they could be swapped for any other traceability technique by adjusting the corresponding skill. Embedded

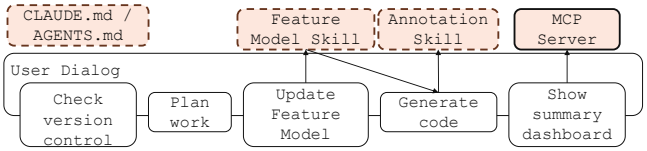


Figure 2: One Agent-HAnS cycle showing the stages the agent executes and involved components per step

feature annotations maintain traceability between implementation assets and feature models at three levels of granularity, illustrated in Fig. 1. *Folder mappings* ① map a folder to features by placing a `.feature-to-folder` file inside it. *File mappings* ② map whole files to features by placing a `.feature-to-file` file. *Fragment mappings* ③ use code comments to mark blocks of code and single lines. All mappings reference entries in a `.feature-model` tree [31].

3 Agent-HAnS

We built Agent-HAnS on the MCP [14]. It attaches to any MCP-capable agentic IDE, such as Claude Code, OpenCode, or Copilot CLI through a project-root instruction file (`AGENTS.md` or `CLAUDE.md`) and an MCP server that collects feature-level data from the agent and serves it to the user via a dashboard. Once installed, the developer engages with Agent-HAnS only through ordinary prompts to the coding agent. The instruction file is read at session start and directs the agent to maintain the feature model and embedded feature annotations using agent skills before and after each coding cycle. The whole dialogue between developer and agent, including the updated feature model and change summary after each cycle, is stored in the agent's context memory. Agent-HAnS comprises five components: a workflow specification, two agent skills, an MCP server and a dashboard. Together, they update the feature model, annotations, and dashboard without further developer action between cycles.

Agentic vs. Manual Annotation. HAnS [22] provides IDE tooling, such as code completion and navigation, that makes it easier for a developer to add and edit embedded feature annotations, but writing the annotation is still a manual action the developer takes while writing code, at a cost measured directly in developer attention [20]. Agent-HAnS instead integrates annotation into the agentic coding workflow itself, so the agent produces the annotation as a native step of code generation, with no separate annotation action from the developer. Correctness can no longer be assumed from the annotator's own intent, which is why Agent-HAnS pairs generation-time annotation with the dashboard review step below.

Workflow Specification. The whole workflow of one coding cycle is specified in the instruction file and visualized in Fig. 2. After each developer prompt, it directs the agent to verify the worktree, read the current feature model via the MCP Server, use the Feature Model Skill to update the `.feature-model`, use the Annotation Skill to annotate generated code, and submit the updated model together with a change summary to the MCP Server.

Feature Model Skill. The Feature Model Skill comprises guidelines for updating the feature model. It specifies the `.feature-model` notation [31], including naming style, indentation for parenthood, and rules for expressing add, rename, and remove deltas, so the agent produces a valid model on every cycle. It addresses Challenge 1 by specifying a fixed format for maintaining feature models.

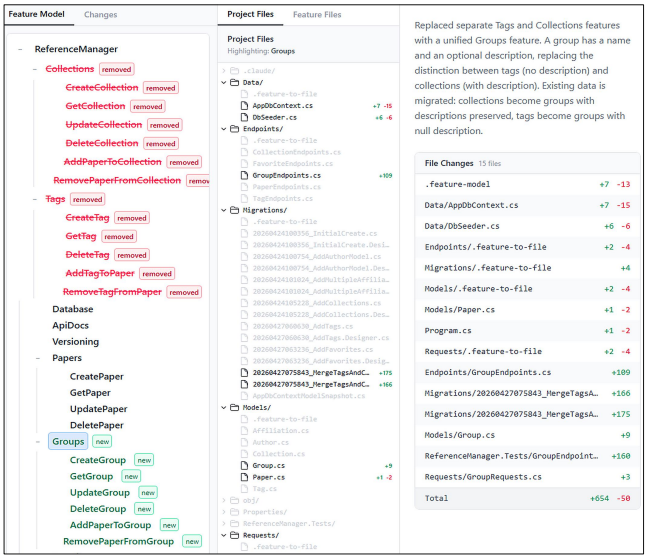


Figure 3: Agent-HAnS dashboard. Left: feature tree with change indicators. Center: Project Files of the selected feature. Right: summary with line additions and deletions.

Annotation Skill. The Annotation Skill specifies the embedded feature annotation notation [31] and is used when writing or modifying source files. Folder mappings are written into the file `.feature-to-folder`, file mappings written in `.feature-to-file`, and fragment mappings directly in code as comments (Fig. 1), with comment-syntax conventions per language and feature-name resolution rules. It addresses Challenge 2 by binding annotation creation to the same step that produces the code.

MCP Server. The MCP Server acts as backend for Agent-HAnS's workflow. After each cycle, it receives feature model updates and change summaries from the agent, runs a `git diff` on the worktree, maps features to the file tree by resolving feature annotations, and stores the result as a JSON payload under a unique cycle id. It then opens the dashboard for that cycle to visualize the changes.

Dashboard. The dashboard (Fig. 3 and Fig. 4) supports the human-in-the-loop review described in Sect. 2. It allows the developer to review changes at the feature-level before issuing the next prompt and browse specific files when a deviation from the expected behavior needs code-level inspection. Changes to the feature model are highlighted in the feature tree, which can be selected. The files affected by changes to the selected feature are listed in the Project Files tab, and a git-style diff summary shows line additions and deletions per file. The Feature Files tab narrows the view to those files belonging to the selected feature. The dashboard also reports the total lines added and removed across the full cycle, giving a session-level overview alongside the per-file breakdown. The Changes tab provides an LLM-generated change summary for each feature, where each feature's purpose is described. Clicking a file in either the Project Files or Feature Files tab opens its content inline, giving the developer code-level access within the dashboard.

4 Demonstration

We demonstrate Agent-HAnS on *ReferenceManager*, a C# /.NET 9 web API for managing research paper references, developed entirely

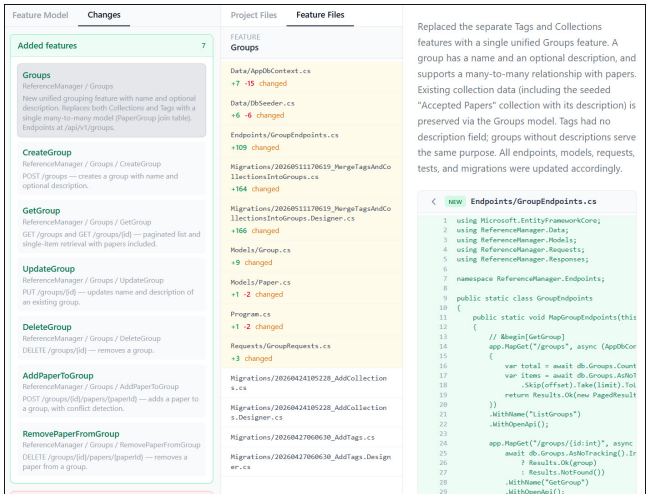


Figure 4: Agent-HAnS dashboard (continued). Left: Changes tab with feature descriptions. Center: Feature Files for the selected feature. Right: file content for code inspection.

through agentic coding using Claude Code with Claude Sonnet 4.6. The project begins as an empty .NET 9 scaffold with no features and no annotations. Agent-HAnS is active throughout the whole process, producing every feature and every annotation. The dataset, including code and prompt transcripts at each step, is available online [19]. The project evolves via 10 evolution steps, each completing one structural feature model change through one to six iterative prompts. In total, the 27 prompts produce 44 C# files and 2,971 lines of application code. After each coding cycle, Agent-HAnS's workflow triggers automatically, updating the feature model and annotating the codebase without developer intervention.

Table 1 summarizes the evolution steps and the feature model changes. The sequence exercises the full range of structural changes of feature models: feature additions, a merge, a split, and a removal. The final model contains 25 features across six top-level features.

Scenario. We focus on Evolution 5, where the features `Tags` and `Collections` are merged into feature `Groups` with a single prompt:

> *Tags and Collections overlap — both associate papers with named groups. Merge them into a single Groups feature. A group has a name and an optional description. Papers can belong to multiple groups and a group can contain many papers. Replace the separate Tags and Collections controllers, models, and join tables with a single Groups controller and model. Migrate existing tags as groups without descriptions and existing collections as groups with their descriptions preserved.*
>
> Developer prompt (Evolution 5)

The LLM removes the `Tags` and `Collections` controllers, models, and migrations. It introduces a `Groups` module with four CRUD and two paper-association sub-features, producing four new source files and a database migration consolidating the two join tables with 287 lines added and 388 removed across the cycle (Table 1).

Automatic Update. Using the Feature Model Skill, the agent reads the modified files alongside the feature model and emits a revised model in which `Tags` and `Collections` are replaced by `Groups` with its six sub-features. The agent then rewrites the `.feature-to-file` mappings for Models, Endpoints, and Requests

and updates the Migrations mapping to include the new migration using the Annotation Skill. In three files, the agent replaces the `Collections` and `Tags` annotation blocks with `Groups`, keeping every fragment annotation up to date. The MCP Server receives the updated model and change summary, then refreshes the dashboard.

Developer View. The dashboard (Fig. 3), shows `Groups` replacing `Tags` and `Collections` in the feature tree. The developer can now inspect the changes at the feature-level by browsing related files.

5 Evaluation

To evaluate Agent-HAnS, we audit the correctness of the ReferenceManager feature model and traceability links (Sect. 4), complemented by a six-application feature-model benchmark and a user study from our companion study [4].

Feature Model Correctness. The final feature model contains 25 features organized in six top-level features. `Authors` received a dedicated `ListAuthors` feature alongside its five CRUD siblings, while `Papers` and `Groups` never received equivalent features of similar granularity, despite both exposing the same kind of list-all endpoint. The defect is not missing detail but consistency, which directly causes two of three fragment-placement problems we observed.

In a companion study [4], we graded Agent-HAnS-generated feature models for six JavaScript/React applications against manually defined baselines, where Agent-HAnS averaged 5.72 on a nine-point scale, ahead of the 5.11 from post-hoc extraction, with models erring toward too much granularity, suggesting granularity is a fragile dimension of automated feature modeling.

Annotation and Mapping Correctness. We manually audited every annotation in the final artifact, checking both whether the referenced feature exists and whether the annotated code matches it, with the full audit included in the dataset.[1] Agent-HAnS produced no folder mappings in this demonstration, so the audit covers the file and fragment granularities it actually used. All 36 `begin`/`end` blocks and 14 line annotations are balanced and, together, cover all 25 features in the model, but only 33 of the 36 blocks (92%) are correctly scoped. One `Groups` block in `DbSeeder.cs` annotates `Paper` seed data rather than group-creation code, and two `GetGroup`/`GetPaper` blocks silently absorb the list-all endpoint because of the model inconsistency above. File-level mappings are less reliable still, with 14 of the 20 file-to-feature mappings (70%) across the seven `.feature-to-file` files correct and six stale. Four dangling mappings in the `Migrations` folder still point migration files at `Tags` and `Collections`, both removed by the Evolution 5 merge, and two mappings in `Models` still point `Author.cs` and `Affiliation.cs` at `Papers`, although the Evolution 8 split promoted `Authors` to a top-level feature. Checking recall rather than precision surfaces a fourth problem, the Evolution 8 split migration itself and the schema migration backing `ImportBibtex` implement features yet carry no annotation at all. All four problems, an under-grown model, stale file mappings, over-broad fragment scope, and untagged schema migrations, concentrate at structural-change steps rather than additive ones, exactly the class of error the dashboard review step is meant to catch, motivating improved merge- and split-detection in Agent-HAnS.

Table 1: ReferenceManager evolution steps and feature model changes tracked by Agent-HAnS

Ev.	Feature model change	Type	+ LoC	− LoC
1	+ `Database`, `ApiDocs`, `Papers`	Add	157	–
2	+ `Collections`, `Versioning`	Add	776	75
3	+ `Tags`	Add	115	–
4	+ `Favorites`	Add	40	–
5	`Tags+Collections` → `Groups`	Merge	287	388
6	+ `Papers.ImportBibTex`	Add	395	–
7	+ `Papers.ExportBibTex`	Add	111	5
8	`Papers.Authors` → `Authors`	Split	474	119
9	+ `Papers.SearchPapers`	Add	112	10
10	− `Favorites`	Remove	1	45

[1] https://johan.martinson.phd/agentic-traceability-maintenance-dataset/audit.html

User Study. In our companion study [4], 14 developers completed coding tasks with and without Agent-HAnS in a within-subjects design, and Agent-HAnS scored 73.9 on the System Usability Scale [9], in the "good" range [3]. Perceived workload on the NASA-TLX was statistically indistinguishable with and without the tool, although physical demand was significantly higher with it present ($p = 0.01$), plausibly from navigating the dashboard, while mental demand and frustration both trended lower. The lowest-rated survey item, whether developers would want to use Agent-HAnS frequently, suggests usability alone does not yet guarantee adoption, possibly because feature models are still unfamiliar to developers. The study found no significant effect on comprehension of, or trust in, the generated code, plausibly because its hour-long tasks produced projects small enough to grasp unaided. Whether these benefits materialize on larger projects remains for a longer-term study.

Agent-HAnS keeps most of its generated feature model and annotations correct, and early evidence suggests the workflow is usable, but model and mappings degrade at structural-change steps, merges and splits, exactly where stronger safeguards are needed. Key finding

6 Related Work

Feature Annotation and Visualization Tools. HAnS [22] provides IDE editing support for embedded feature annotations, enabling developers to mark code fragments and files as they write code. FeatRacer [24] assists semi-automated feature location by recommending feature-to-code mappings when committing. HAnS-viz [21] visualizes feature models and feature characteristics such as scattering and tangling [25]. Hermann et al. [13] extend HAnS-viz with a feature evolution timeline over a repository's history. Previous tools target manual coding workflows, where a human writes code and adds annotations. Agent-HAnS moves annotation into the generation step itself, making it a native responsibility of the coding agent. To our knowledge, Agent-HAnS is the first tool to maintain a feature model and traceability links during agentic code generation.

LLMs and Agentic Tooling for Feature Location. Acher and Martinez [1] apply LLMs to reengineer software variants into product lines, though human review is necessary for acceptable results. Stümpfle et al. [32] extend this with feedback loops in an agentic setting. These works produce a one-shot feature model without traceability, rather than maintaining both during code generation. *Specification-driven development* formalizes developer intent as a machine-checkable artifact before generation begins. Feeding such

specifications into the Feature Model Skill is a promising way to improve annotation practicability. Code comprehension tools such as call-graph explorers expose language-level structure post-hoc, whereas Agent-HAnS records feature-level intent at generation time.

7 Conclusion

We presented Agent-HAnS, an agentic coding workflow which maintains a feature model and generates embedded feature annotations throughout software evolution by updating them after each code-generation step. Within 27 prompts and 10 steps, Agent-HAnS maintained feature traceability for 25 features across 44 generated C#/.NET 9 files across four evolution types, with 92% of fragment annotations and 70% of file mappings correct and errors concentrating at merge and split steps (Sect. 5).

We plan to extend Agent-HAnS with a feature-scope selection interface and a session-persistent timeline view to view feature evolution history across sessions, improve merge and split detection, conduct a longer-term user study measuring comprehension gains, extend our correctness audit to more LLM families and real-world projects, and evaluate token-cost overhead and resilience to hallucinated feature-model updates. Agent-HAnS depends only on the MCP interface and a plain project-root instruction file, both agent-agnostic, and its skills are plain specifications of the embedded annotation notation rather than agent-specific prompts, so the approach does not bet on any single coding agent's continued existence. Lifting feature structure into the generation loop is a step toward feature-oriented software evolution in agent-built systems.

Data Availability Statement

Agent-HAnS and the evolution dataset are released as open-source software, archived on Zenodo [14, 19].

References

[1] Mathieu Acher and Jabier Martinez. 2023. Generative AI for Reengineering Variants into Software Product Lines: An Experience Report. In *VariVolution*. doi:10.1145/3579028.3609016

[2] Sven Apel and Christian Kästner. 2009. An overview of feature-oriented software development. *J. Object Technol.* 8, 5 (2009), 49–84. doi:10.5381/jot.2009.8.5.c5

[3] Aaron Bangor, Philip Kortum, and James Miller. 2009. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of usability studies* 4, 3 (2009), 114–123. https://dl.acm.org/doi/abs/10.5555/2835587.2835589

[4] Jacob Bengtsson and Moltas Hultin. 2026. *Agent HAnS: Using feature traceability to improve comprehension and trust in LLM-based coding.* Master's thesis. Chalmers University of Technology and University of Gothenburg.

[5] Thorsten Berger, Daniela Lettner, Julia Rubin, Paul Grünbacher, Adeline Silva, Martin Becker, Marsha Chechik, and Krzysztof Czarnecki. 2015. What is a feature? A qualitative study of features in industrial software product lines. In *SPLC*. doi:10.1145/2791060.2791108

[6] Thorsten Berger, Wardah Mahmood, Ramzi Abu Zahra, Igor Vassilevski, Andreas Burger, Wenbin Ji, Michal Antkiewicz, and Krzysztof Czarnecki. 2025. Cost and Benefit of Tracing Features with Embedded Annotations. *Trans. Softw. Eng. Methodol.* 35, 4 (2025), 109:1–109:47. doi:10.1145/3746060

[7] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2023. Taking flight with copilot. *Commun. ACM* 66, 6 (2023), 56–62. doi:10.1145/3589996

[8] Jan Bosch. 2000. *Design and Use of Software Architectures: Adopting and Evolving a Product-Line Approach.* ACM. https://dl.acm.org/doi/book/10.5555/339362

[9] John Brooke. 1995. SUS: A quick and dirty usability scale. *Usability Eval. Ind.* 189 (11 1995). doi:10.1201/9781498710411-35

[10] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: a taxonomy and survey. *J. Softw.: Evol. Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567

[11] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2025. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook–a Grey Literature Review. *arXiv:2510.00328* (2025). doi:10.1145/3786583.3786866

[12] Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. 2025. Agentic Software Engineering: Foundational Pillars and a Research Roadmap. *arXiv preprint arXiv:2509.06216* (2025). doi:10.48550/arXiv.2509.06216

[13] Kevin Hermann, Johan Martinson, and Thorsten Berger. 2025. Visualizing Feature-Oriented Software Evolution. In *SPLC, tools track*. doi:10.1145/3744915.3748471

[14] Moltas Hultin, Jacob Bengtsson, and Johan Martinson. 2026. Agent-HAnS ASE26. Software, version 0.0.1-ASE26. doi:10.5281/zenodo.21772184

[15] Wenbin Ji, Thorsten Berger, Michal Antkiewicz, and Krzysztof Czarnecki. 2015. Maintaining feature traceability with embedded annotations. In *SPLC*. doi:10.1145/2791060.2791107

[16] Kyo Kang, Sholom Cohen, James Hess, William Novak, and A. Peterson. 1990. *Feature-Oriented Domain Analysis (FODA) Feasibility Study.* Technical Report CMU/SEI-90-TR-021. SEI, Carnegie Mellon University.

[17] Jacob Krüger, Thorsten Berger, and Thomas Leich. 2019. *Features and How to Find Them: A Survey on Manual Feature Location.* LLC/CRC Press. doi:10.1201/9780429022067-9

[18] Stephane Maes. 2025. The Gotchas of AI Coding and Vibe Coding. It's All About Support And Maintenance. doi:10.31219/osf.io/kjz9t_v1

[19] Johan Martinson. 2026. Agent-HAnS ASE26 Demonstration Dataset. Dataset, version 1.0.0. doi:10.5281/zenodo.21771987

[20] Johan Martinson, Kevin Hermann, and Thorsten Berger. 2026. Overhead of Recording Feature Locations with Embedded Annotations. In *VARIABILITY*.

[21] Johan Martinson, Kevin Hermann, Riman Houbbi, David Stechow, and Thorsten Berger. 2025. Lightweight Visualization of Software Features with HAnS-viz. In *SPLC, tools track*. doi:10.1145/3748269.3748487

[22] Johan Martinson, Herman Jansson, Mukelabai Mukelabai, Thorsten Berger, Alexandre Bergel, and Truong Ho-Quang. 2021. HAnS: IDE-based editing support for embedded feature annotations. In *Proceedings of the 25th ACM International Systems and Software Product Line Conference-Volume B*. 28–31. doi:10.1145/3461002.3473072

[23] Christian Meske, Tobias Hermanns, Esther von der Weiden, Kai-Uwe Loser, and Thorsten Berger. 2025. Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda. *IEEE Access* 13 (2025), 213242–213259. doi:10.1109/access.2025.3645466

[24] Mukelabai Mukelabai, Kevin Hermann, Thorsten Berger, and Jan-Philipp Steghöfer. 2023. FeatRacer: Locating Features Through Assisted Traceability. *TSE* 49, 12 (2023), 5060–5083. doi:10.1109/tse.2023.3324719

[25] Leonardo Passos, Jesus Padilla, Thorsten Berger, Sven Apel, Krzysztof Czarnecki, and Marco Tulio Valente. 2015. Feature Scattering in the Large: A Longitudinal Study of Linux Kernel Device Drivers. In *14th International Conference on Modularity (MODULARITY)*. doi:10.1145/2724525.2724575

[26] Denys Poshyvanyk, Yann-Gael Gueheneuc, Andrian Marcus, Giuliano Antoniol, and Vaclav Rajlich. 2007. Feature Location Using Probabilistic Ranking of Methods Based on Execution Scenarios and Information Retrieval. *IEEE Transactions on Software Engineering* 33, 6 (2007), 420–432. doi:10.1109/TSE.2007.1016

[27] Francisca Pérez, Jorge Echeverría, Raúl Lapeña, and Carlos Cetina. 2020. Comparing manual and automated feature location in conceptual models: A Controlled experiment. *Inf. Softw. Technol.* 125 (2020). doi:10.1016/j.infsof.2020.106337

[28] Partha Pratim Ray. 2025. A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions. *Authorea Preprints* (2025). doi:10.36227/techrxiv.174681482.27435614/v1

[29] Julia Rubin and Marsha Chechik. 2013. *A Survey of Feature Location Techniques.* Springer Berlin Heidelberg, Berlin, Heidelberg, 29–58. doi:10.1007/978-3-642-36654-3_2

[30] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. 2025. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI. *arXiv preprint arXiv:2505.19443* (2025). doi:10.48550/arXiv.2505.19443

[31] Tobias Schwarz, Wardah Mahmood, and Thorsten Berger. 2020. A Common Notation and Tool Support for Embedded Feature Annotations. In *SPLC*. doi:10.1145/3382026.3431253

[32] Johannes Stümpfle, Devansh Atray, Nasser Jazdi, and Michael Weyrich. 2025. Large Language Model assisted Transformation of Software Variants into a Software Product Line. In *ICSR*. doi:10.1109/icsr66718.2025.00008

[33] Jinshui Wang, Xin Peng, Zhenchang Xing, and Wenyun Zhao. 2013. How developers perform feature location tasks: a human-centric and process-oriented exploratory study. *J. Softw.: Evol. Process* 25, 11 (2013), 1193–1224. doi:10.1002/smr.1593

[34] Andrzej Wąsowski and Thorsten Berger. 2023. Feature Modeling. In *Domain-Specific Languages: Effective Modeling, Automation, and Reuse*. Springer, 437–457. doi:10.1007/978-3-031-23669-3_12

[35] Yangtian Zi, Luisa Li, Arjun Guha, Carolyn Anderson, and Molly Q Feldman. 2025. "I Would Have Written My Code Differently': Beginners Struggle to Understand LLM-Generated Code. In *FSE*. doi:10.1145/3696630.3731663